

AI-Based Interview Question Generator & Assessment Platform

Technical reference, architecture guide and user manual

Attribute	Detail
Project type	Interactive technical-interview simulation and assessment engine
Frontend / UI	Streamlit (multi-page web application)
AI provider	Google Gemini (free-tier models), with an OpenAI alternative evaluator
Persistence	None - session-state memory only (zero-database architecture)
Language / runtime	Python 3.10+
License	MIT
Source	Derived from the project README.md

Table of Contents

1. Executive Summary	2
1.1 Intended Audience	2
2. Key Features	2
2.1 Role-Specific Question Generation	2
2.2 Gemini Integration with Free-Tier Resiliency	2
2.3 Interactive Live Interview Simulation	2
2.4 Rubric-Based Evaluation	3
2.5 Analytics and Executive Reports	3
2.6 Zero-Database Architecture	3
3. System Architecture	3
3.1 Workflow Stages	4
3.2 Design Principles	4
4. Project Structure and Components	4
4.1 Module Reference	5
5. Installation and Setup	5
5.1 Prerequisites	5
5.2 Step-by-Step	5
6. Configuration Reference	6
6.1 Environment Variables	6
7. Usage Guide	6
7.1 Running an Interview	6
7.2 Evaluation Output Fields	7
7.3 Resiliency Behaviour	7
8. Testing	7
9. Limitations and Considerations	8
10. Documentation Review Notes	8
11. Glossary	9

1. Executive Summary

The **AI-Based Interview Question Generator & Assessment Platform** is a Streamlit application that uses Google Gemini to run an end-to-end technical interview: it generates role-specific questions, presents them to a candidate one at a time, scores each answer against a rubric, and finally compiles an analytical performance report.

The system is designed around three ideas:

- **Tailoring** - questions adapt to job role, experience level, skills, interview type and difficulty.
- **Objectivity** - answers are scored 0-10 against concrete technical criteria, with explicit lists of correct and missing points.
- **Resilience** - if the free-tier Gemini quota is exhausted, the platform silently falls back to a built-in generator and heuristic evaluator so the interview is never interrupted.

It requires no database: all state lives in the Streamlit session, which keeps deployment trivial but means results are not stored permanently.

1.1 Intended Audience

Audience	How they use the platform
Candidates	Practise technical interviews, receive instant feedback and targeted revision topics.
Hiring managers / recruiters	Run structured screening sessions and export a summary report.
Trainers and educators	Assess learners across multiple domains and difficulty levels.
Developers / contributors	Extend question generation, evaluation logic, or reporting.

2. Key Features

2.1 Role-Specific Question Generation

- **Domains:** Frontend, Backend, Full Stack, ML/AI, DevOps, Data Analysis and Custom roles.
- **Experience levels (5):** Fresher, 0-1 yrs, 1-3 yrs, 3-5 yrs, 5+ yrs.
- **Difficulty tiers (3):** Easy, Medium, Hard.
- **Question categories:** Technical, Conceptual, Scenario-Based, Problem Solving and Behavioral.

2.2 Gemini Integration with Free-Tier Resiliency

The platform is configured for free-tier Gemini models: `gemini-2.0-flash`, `gemini-1.5-flash`, `gemini-1.5-flash-8b`, `gemini-2.0-flash-lite` and `gemini-2.5-flash`. When an unpaid key reaches its rate limit (HTTP 429) or quota, the engine automatically switches to a built-in dynamic question template and a heuristic evaluator.

2.3 Interactive Live Interview Simulation

- Sequential, one-question-at-a-time pacing that mimics a real interview room.
- Real-time response capture and answer-review tabs.
- Automated follow-up probing questions generated from the candidate's answers.

2.4 Rubric-Based Evaluation

- Strict 0-10 scoring against concrete technical criteria.
- Structured output: `correct_points` (demonstrated strengths), `missing_points` (omissions), concise feedback and actionable improvement steps.

2.5 Analytics and Executive Reports

- Score percentage, performance tier and topic-mastery breakdown.
- Weak-topic identification with recommended revision areas.
- Exportable summary for candidates and hiring managers.

2.6 Zero-Database Architecture

Pure session-state memory management with no SQL or NoSQL dependency, minimising setup and operational overhead.

3. System Architecture

The platform is a linear pipeline with a feedback loop between the live session and the evaluator, and a fallback branch that protects the pipeline from API quota failures.

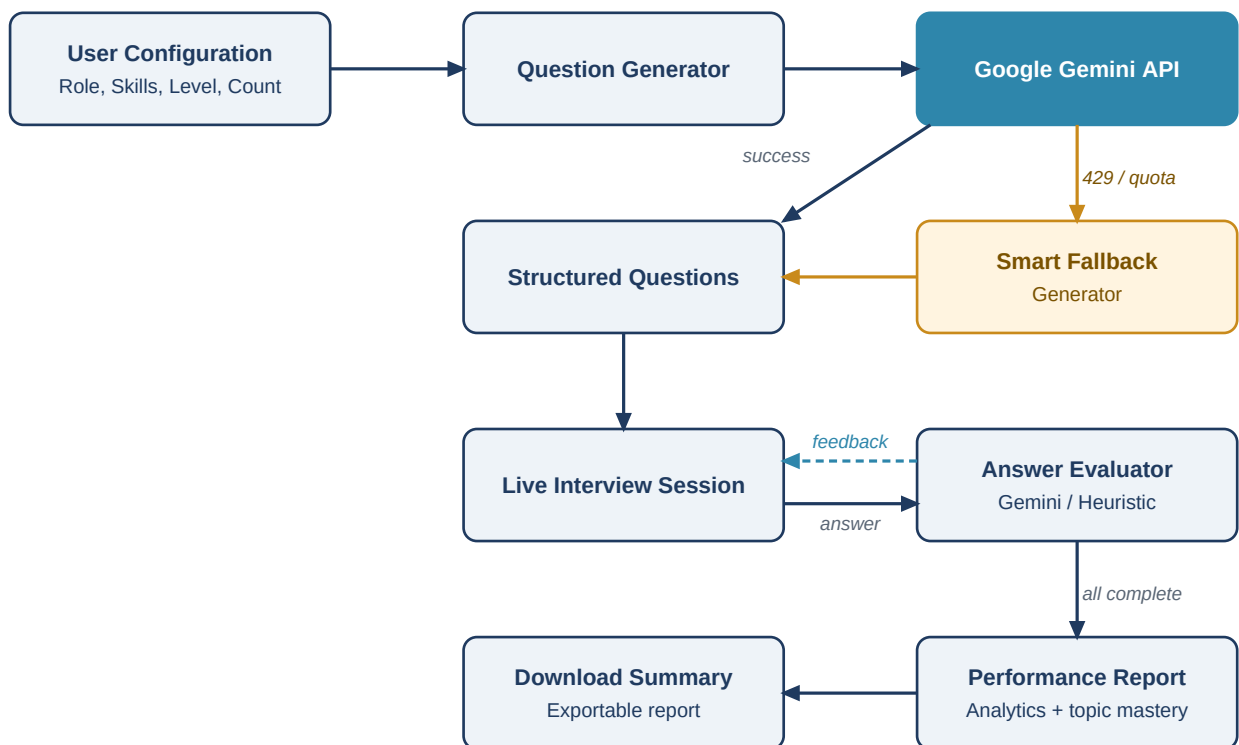


Figure 1 - End-to-end workflow. Dashed arrow shows the per-answer evaluation loop; the amber path is the quota fallback.

3.1 Workflow Stages

#	Stage	Description
1	User Configuration	The user selects role, skills, experience, interview type, difficulty and question count.
2	Question Generation	The generator sends a structured request to Gemini and parses the returned questions (with rubrics).
3	Fallback (conditional)	On HTTP 429 or quota exhaustion, a built-in dynamic template generator supplies questions instead.
4	Live Interview	Questions are shown sequentially; the candidate submits an answer for each.
5	Answer Evaluation	Gemini (or the heuristic engine) returns a score, rubric feedback and a follow-up question.
6	Report Generation	After the last question, scores are aggregated into analytics and topic mastery.
7	Export	The candidate or hiring manager downloads the summary.

3.2 Design Principles

- **Graceful degradation:** quota failures never stop an interview.
- **Structured data contracts:** LLM output is requested as JSON, validated, and self-repaired if malformed.
- **Separation of concerns:** UI, generation, evaluation and reporting live in separate modules.
- **Statelessness at rest:** no external persistence layer.

4. Project Structure and Components

```
HCL/
|-- app.py                      # Main Streamlit application and multi-page UI
|-- services/
|   |-- __init__.py            # Service package exports
|   |-- question_generator.py  # Gemini API caller & dynamic question generator
|   |-- evaluator.py           # Rubric-based answer evaluation service
|   |-- report_generator.py    # Analytics & report calculation engine
|   |-- answer_evaluator.py    # Alternative OpenAI evaluation service
|-- tests/
|   |-- test_qa_suite.py       # 12-point automated test suite
|-- .env.example               # Template for environment variables
|-- requirements.txt           # Python package dependencies
|-- .gitignore                 # Git ignore rules
-- README.md                   # Project documentation
```

4.1 Module Reference

File	Responsibility	Key behaviour
app.py	Streamlit entry point; renders configuration, interview and report pages.	Manages session state, page navigation and answer submission.
services/question_generator.py	Calls Gemini and produces structured questions.	Handles rubric scaling, invalid-JSON repair, and quota fallback to templates.
services/evaluator.py	Scores answers against rubrics.	Clamps scores to 0-max; returns correct_points, missing_points, feedback, improvements and follow-up.
services/report_generator.py	Aggregates results.	Computes percentage, performance tier, topic breakdown and weak topics.
services/answer_evaluator.py	Alternative evaluator using OpenAI.	Provides a non-Gemini scoring path.
tests/test_qa_suite.py	Automated verification.	12 unit and integration checks (see Section 8).
.env.example	Environment template.	Documents required API key variables.
requirements.txt	Dependency manifest.	Installed via pip.

5. Installation and Setup

5.1 Prerequisites

- Python **3.10 or later**.
- A **Google Gemini API key** - available free from Google AI Studio (aistudio.google.com).

5.2 Step-by-Step

Step 1 - Navigate to the project (replace with your own path):

```
cd "<path-to-project>/HCL"
```

Step 2 - Create a virtual environment (optional but recommended):

```
python -m venv .venv

# Windows (PowerShell)
.\.venv\Scripts\Activate.ps1

# Linux / macOS
source .venv/bin/activate
```

Step 3 - Install dependencies:

```
pip install -r requirements.txt
```

Step 4 - Configure the API key. Create a `.env` file in the project root (or copy `.env.example`):

```
GEMINI_API_KEY=your_actual_gemini_api_key_here
```

GOOGLE_API_KEY or OPENAI_API_KEY may also be used as fallback variable names.

Step 5 - Launch the app:

```
streamlit run app.py
```

Then open `http://localhost:8501` in a browser.

6. Configuration Reference

Parameter	Options / Format	Effect
Job Role	Frontend, Backend, Full Stack, Python, ML, DevOps, Data Analyst, Custom	Sets the domain context for question targeting.
Experience	Fresher, 0-1 yrs, 1-3 yrs, 3-5 yrs, 5+ yrs	Calibrates difficulty and technical depth.
Skills	Python, FastAPI, React, SQL, Docker, TypeScript, etc. + custom entries	Focuses questions on specific tools and libraries.
Interview Type	Technical, Behavioral, Coding, System Design, Mixed	Determines question structure and style.
Difficulty	Easy, Medium, Hard	Scales rubric strictness and scenario depth.
Question Count	5 to 15	Controls interview length.

6.1 Environment Variables

Variable	Purpose	Priority
GEMINI_API_KEY	Primary Google Gemini credential.	Primary
GOOGLE_API_KEY	Alternative name for a Google key.	Fallback
OPENAI_API_KEY	Credential for the OpenAI-based evaluator.	Fallback / alternative

Security: never commit `.env` to version control. The project ships a `.gitignore` for this purpose.

7. Usage Guide

7.1 Running an Interview

1. **Configure** - choose role, experience, skills, interview type, difficulty and number of questions (5-15).
2. **Generate** - start the interview; questions are generated by Gemini or, if quota is exhausted, by the fallback generator.
3. **Answer** - respond to each question in turn. Empty or whitespace-only answers are handled explicitly.
4. **Review feedback** - each answer receives a 0-10 score, strengths, omissions, feedback, improvement steps and a possible follow-up question.
5. **View the report** - after the last question, see the percentage score, performance tier, topic mastery and weak topics.
6. **Export** - download the report summary.

7. **Reset** - clear the session state to begin a new interview.

7.2 Evaluation Output Fields

Field	Meaning
score	Integer/decimal from 0 to the question maximum (clamped).
correct_points	Technical points the candidate demonstrated correctly.
missing_points	Expected points that were omitted or incorrect.
feedback	Concise written assessment.
improvement_steps	Actionable recommendations for the candidate.
follow-up	Probing question derived from the candidate's answer.

7.3 Resiliency Behaviour

Condition	System response
Valid Gemini response	Structured questions / evaluations are used directly.
Invalid or malformed JSON	Self-repair logic attempts to recover a valid structure.
HTTP 429 or quota exhausted	Automatic switch to built-in question templates and heuristic evaluator.
Empty candidate answer	Handled by a dedicated validation path rather than sent for scoring.

8. Testing

The automated suite verifies twelve core components. Run it with:

```
python -m unittest tests/test_qa_suite.py -v
```

#	Area	What is verified
1	Startup and constants	Application starts; constants are intact.
2	Configuration generation	Interview configuration dictionary is built correctly.
3	Input validation	Boundary and edge-case inputs are handled.
4	Question generation	LLM question generation and rubric scaling.
5	Invalid JSON handling	Malformed responses are detected and repaired.
6	Empty answers	Empty / whitespace submissions are handled.
7	Evaluation logic	Scores are clamped between 0 and the maximum.
8	Follow-up tracking	Follow-up questions are tracked correctly.
9	Score calculation	Totals and percentages are accurate.
10	Report metrics	Final report and topic breakdown are correct.
11	State reset	Session state clears fully.
12	Zero-database constraint	No database dependency is introduced.

9. Limitations and Considerations

- **No persistence:** refreshing or closing the session discards interview data; export the report before leaving.
- **Free-tier quotas:** heavy use will trigger the fallback, whose questions and heuristic scoring are less nuanced than Gemini's.
- **Heuristic evaluation:** fallback scoring is rule-based and should not be treated as equivalent to LLM-based rubric assessment.
- **Model availability:** Google changes and retires model versions over time; verify that the listed Gemini models are still offered.
- **Single-user sessions:** there is no authentication, multi-user management or candidate history.

10. Documentation Review Notes

While analysing the README, the following items were noticed. They are suggestions for improving the source documentation, not defects confirmed in the code.

Observation	Suggestion
The setup step contains a machine-specific Windows path (a user's OneDrive folder).	Replace with a generic placeholder such as <code>cd HCL</code> , or add a git clone step.
'Interview Type' options (Technical, Behavioral, Coding, System Design, Mixed) differ from the 'Question categories' in the feature list (Technical, Conceptual, Scenario-Based, Problem Solving, Behavioral).	Clarify how the two settings relate, or align the terminology.
The Job Role list in the feature section (Frontend, Backend, Full Stack, ML/AI, DevOps, Data Analysis, Custom) differs slightly from the configuration table (which adds Python).	Use a single canonical list.
<code>answer_evaluator.py</code> (OpenAI) and <code>evaluator.py</code> (Gemini) both exist, but the README does not say which one the app uses.	State the default evaluator and how to switch.
The README states MIT licensing, but no LICENSE file appears in the project structure.	Add a LICENSE file.
No screenshots, sample report, or deployment instructions are included.	Add screenshots and a Streamlit Community Cloud / Docker deployment section.
Test command uses a file path with <code>unittest</code> .	Consider <code>python -m unittest discover tests -v</code> for cross-platform reliability.

11. Glossary

Term	Definition
Streamlit	Python framework for building interactive data and AI web apps.
Google Gemini	Google's family of generative AI models used for question generation and evaluation.
HTTP 429	'Too Many Requests' - the status returned when an API rate limit is exceeded.
Rubric	A set of concrete criteria used to score an answer objectively.
Heuristic evaluator	Rule-based scoring used when the LLM is unavailable.
Session state	In-memory data held by Streamlit for the duration of a user's session.
Topic mastery	Per-topic performance derived from question scores.
Follow-up probing	A dynamic additional question based on the candidate's previous answer.